

Esercizio 1 – RIA

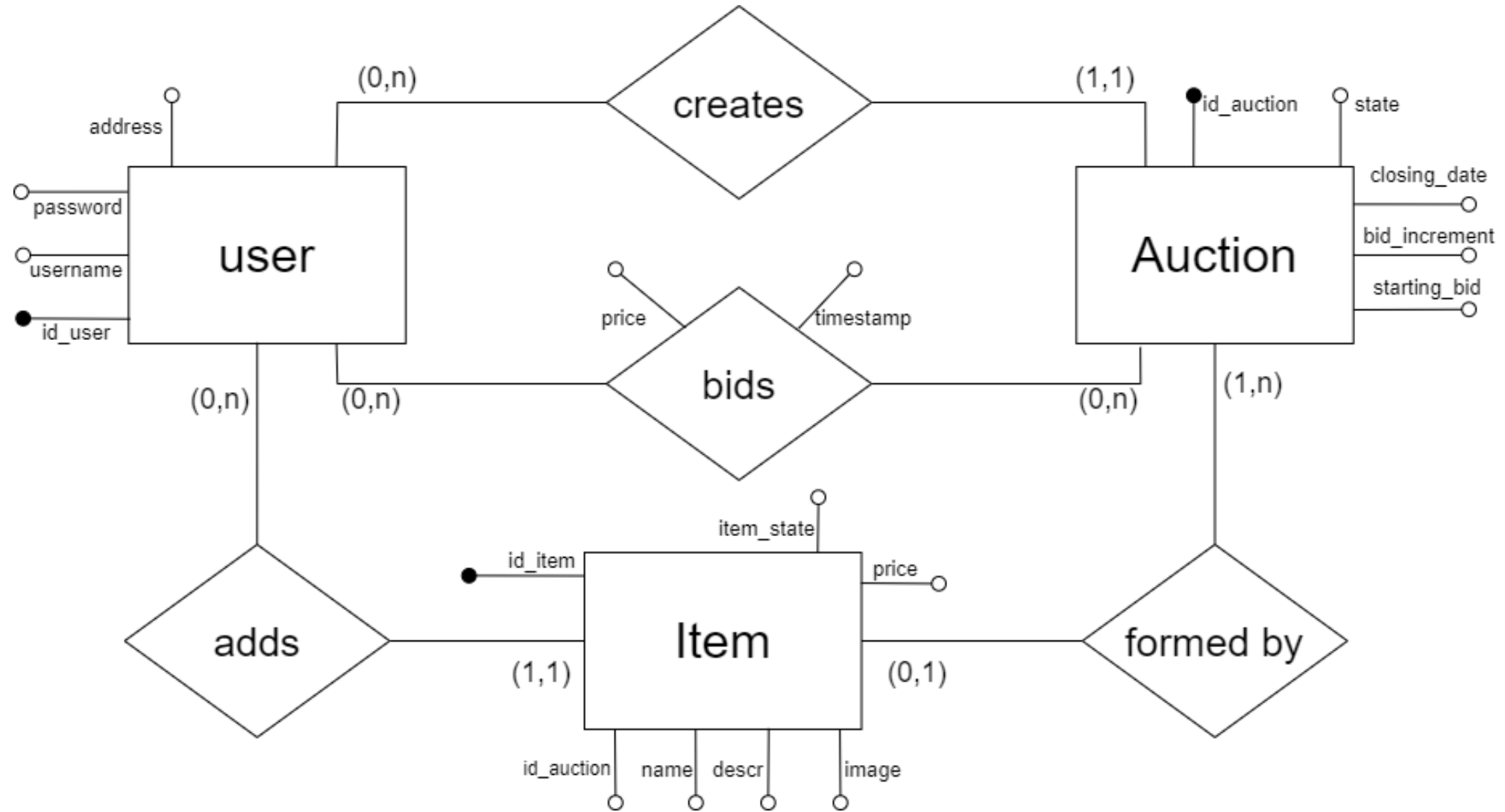
Alberto Biffi – Silvia Firenze

Esercizio 1: Aste Online (Pure HTML) – Analisi DB

- Un'applicazione web consente la gestione di **aste** online. Gli **utenti** accedono tramite login e possono vendere e acquistare all'asta. La HOME page contiene due link, uno per accedere alla pagina VENDO e uno per accedere alla pagina ACQUISTO. La pagina VENDO mostra una lista delle aste create dall'utente e non ancora chiuse, una lista delle aste da **lui** create e chiuse e due form, una per **creare un nuovo articolo** e una per **creare una nuova asta** per vendere gli articoli dell'utente. Il primo form inserisce nuovi articoli nel database e il secondo mostra l'elenco degli articoli disponibili nel database e dà la possibilità di selezionarne più di uno. Un **articolo** ha **codice**, **nome**, **descrizione**, **immagine** e **prezzo**. Un'asta comprende uno o più **articoli** messi in vendita, il prezzo iniziale dell'insieme di articoli, il rialzo minimo di ogni offerta (espresso come un numero intero di euro) e una scadenza (data e ora, es. 19-04-2021 alle 24:00). Il prezzo iniziale dell'asta è ottenuto come somma del prezzo degli articoli compresi nell'offerta. Lo stesso **articolo non può essere incluso in aste diverse**. Una volta venduto, un **articolo non deve essere più disponibile per l'inserimento in ulteriori aste**. La lista delle aste è ordinata per data+ora crescente. L'elenco riporta: codice e nome degli articoli compresi nell'asta, offerta massima, tempo mancante (numero di giorni e ore) tra il momento (data ora) del login e la data e ora di chiusura dell'asta. Cliccando su un'asta compare una pagina DETTAGLIO ASTA che riporta per un'asta aperta tutti i dati dell'asta e la lista delle offerte (**nome utente**, **prezzo offerto**, **data e ora** dell'offerta) ordinata per data+ora decrescente. Un bottone CHIUDI permette all'utente di chiudere l'asta se è giunta l'ora della scadenza (si ignori il caso di aste scadute ma non chiuse dall'utente e non ci si occupi della chiusura automatica di aste dopo la scadenza). Se l'asta è chiusa, la pagina riporta tutti i dati dell'asta, il nome dell'aggiudicatario, il prezzo finale e l'**indirizzo** (fisso) **di spedizione** dell'utente. La pagina ACQUISTO contiene una form di ricerca per parola chiave. Quando l'acquirente invia una parola chiave la pagina ACQUISTO è aggiornata e mostra un elenco di aste aperte (la cui scadenza è posteriore alla data e ora dell'invio) per cui la parola chiave compare nel nome o nella descrizione di almeno uno degli articoli dell'asta. La lista è ordinata in modo decrescente in base al tempo (numero di giorni e ore) mancante alla chiusura. Cliccando su un'asta aperta compare la pagina OFFERTA che mostra i dati degli articoli, l'elenco delle offerte pervenute in ordine di data+ora decrescente e un campo di input per inserire la propria offerta, che deve essere superiore all'offerta massima corrente di un importo pari almeno al rialzo minimo. Dopo l'invio dell'offerta la pagina OFFERTA mostra l'elenco delle offerte aggiornate. La pagina ACQUISTO contiene anche un elenco delle offerte aggiudicate all'utente con i dati degli articoli e il prezzo finale.

- **Entities**, **attributes**, **relationships**

Database design



Logical scheme

User(id_user, username, password, address);

Auction(id_auction, id_user, starting_bid, bid_increment, auction_state, closing_date);

Item(id_item, id_auction, id_user, price, item_state, image, name, descr);

Bid(id_user, id_auction, price, timestamp);

Foreign keys:

Auction.id_user -> User.id_user

Item.id_user -> User.id_user

Bid.id_user -> User.id_user, Bid.id_auction -> Auction.id_auction

Tables definitions

```
CREATE TABLE `auction` (  
  `id_user` int NOT NULL,  
  `id_auction` int NOT NULL AUTO_INCREMENT,  
  `starting_bid` int DEFAULT NULL,  
  `bid_increment` int DEFAULT NULL,  
  `closing_date` date NOT NULL,  
  `auction_state` enum('opened','closed') DEFAULT NULL,  
  PRIMARY KEY (`id_auction`),  
  KEY `id_user` (`id_user`),  
  CONSTRAINT `auction_ibfk_1` FOREIGN KEY (`id_user`)  
REFERENCES `user` (`id_user`)  
) ENGINE=InnoDB
```

```
CREATE TABLE `bid` (  
  `id_user` int NOT NULL,  
  `id_auction` int NOT NULL,  
  `price` int NOT NULL,  
  `timestamp` datetime NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
  KEY `id_user` (`id_user`),  
  KEY `bid_ibfk_2_idx` (`id_auction`),  
  CONSTRAINT `bid_ibfk_1` FOREIGN KEY (`id_user`)  
REFERENCES `user` (`id_user`),  
  CONSTRAINT `bid_ibfk_2` FOREIGN KEY (`id_auction`)  
REFERENCES `auction` (`id_auction`) ON DELETE RESTRICT ON  
UPDATE RESTRICT  
) ENGINE=InnoDB
```

Tables definitions

```
CREATE TABLE `item` (  
  `id_user` int NOT NULL,  
  `id_item` int NOT NULL AUTO_INCREMENT,  
  `id_auction` int DEFAULT NULL,  
  `price` int NOT NULL,  
  `name` varchar(20) NOT NULL,  
  `description` varchar(255) NOT NULL,  
  `img_name` varchar(100) CHARACTER SET utf8mb3 COLLATE  
utf8mb3_general_ci NOT NULL,  
  `item_state` enum('not_auctioned','auctioned','sold')  
DEFAULT NULL,  
  PRIMARY KEY (`id_item`),  
  KEY `id_user` (`id_user`),  
  CONSTRAINT `item_ibfk_1` FOREIGN KEY (`id_user`)  
REFERENCES `user` (`id_user`)  
) ENGINE=InnoDB
```

```
CREATE TABLE `user` (  
  `id_user` int NOT NULL AUTO INCREMENT,  
  `username` varchar(20) NOT NULL,  
  `password` varchar(16) NOT NULL,  
  `address` varchar(100) NOT NULL,  
  PRIMARY KEY (`id_user`),  
  UNIQUE KEY `username` UNIQUE (`username`)  
) ENGINE=InnoDB
```

Server side components:

Model objects (beans):

- User
- Auction
- Item
- Bid

Controllers (servlets):

- Login
- Register
- GetClosedAuctions
- GetOpenedAuctions
- GetStillOpenedAuctions
- GetAuctionBids
- GetAuctionDetails
- CreateAuction
- CreateItem
- CloseAuction
- GetAuctionItems
- GetAwardedAuctions
- GetNotAuctionedItems
- RegisterBid
- SearchByKeyword
- Logout

Data Access Objects (DAO) (first part):

- UserDao:

```
findUser(username,password) : user  
registerUser(username, password, address) : void  
getUserById(id_user) : user  
getUserByUsername(username) : user
```

- AuctionDAO

```
getOpenedAuctionsByUserId(id_user) : list<Auction>  
getClosedAuctionsByUserId(id_user) : list<Auction>  
createAuction(id_user, bid_increment, closing_date, ids_item) : void  
getAuctionDetailsByAuctionId(id_auction) : Auction  
isMyAuction(id_user, id_auction) : boolean  
closeAuction(id_auction) : void  
searchByKeyword(keyword) : list<Auction>  
getBidIncrementByAuctionId(id_auction) : int  
getStartingBidByAuctionId(id_auction) : int  
isClosed(id_auction) : boolean  
canBeClosed(id_auction) : boolean  
getStillOpenedAuction(ids_auction) : String
```

Server side components:

Data Access Objects (DAO) (second part):

- ItemDAO:
 - getItemById(id_item) : item
 - allItemMine(id_user, ids_item) : boolean
 - publishItem(id_user, price, name, description, img_name) : void
 - getNotAuctionedItemById(id_item) : list<Item>
 - getWonItemsByAuctionIds(ids_auction) : list<Item>
 - getItemsByAuctionId(id_auction) : list<Item>
- BidDAO:
 - getBidsByAuctionId(id_auction) : list<Bid>
 - registerBid(id_user, id_auction, price) : void
 - getBuyerIdByAuctionId(id_auction) : int
 - getAwardedAuctionsByUserId(id_user) : list<Bid>
 - getMaxBidByAuctionId(id_auction) : int

Views (Templates):

login.html
home.html

Filters:

- LoggedUser
- NotLoggedUser

Client side components:

Login:

Login form

Register form

(both manage submit and errors)

Home:

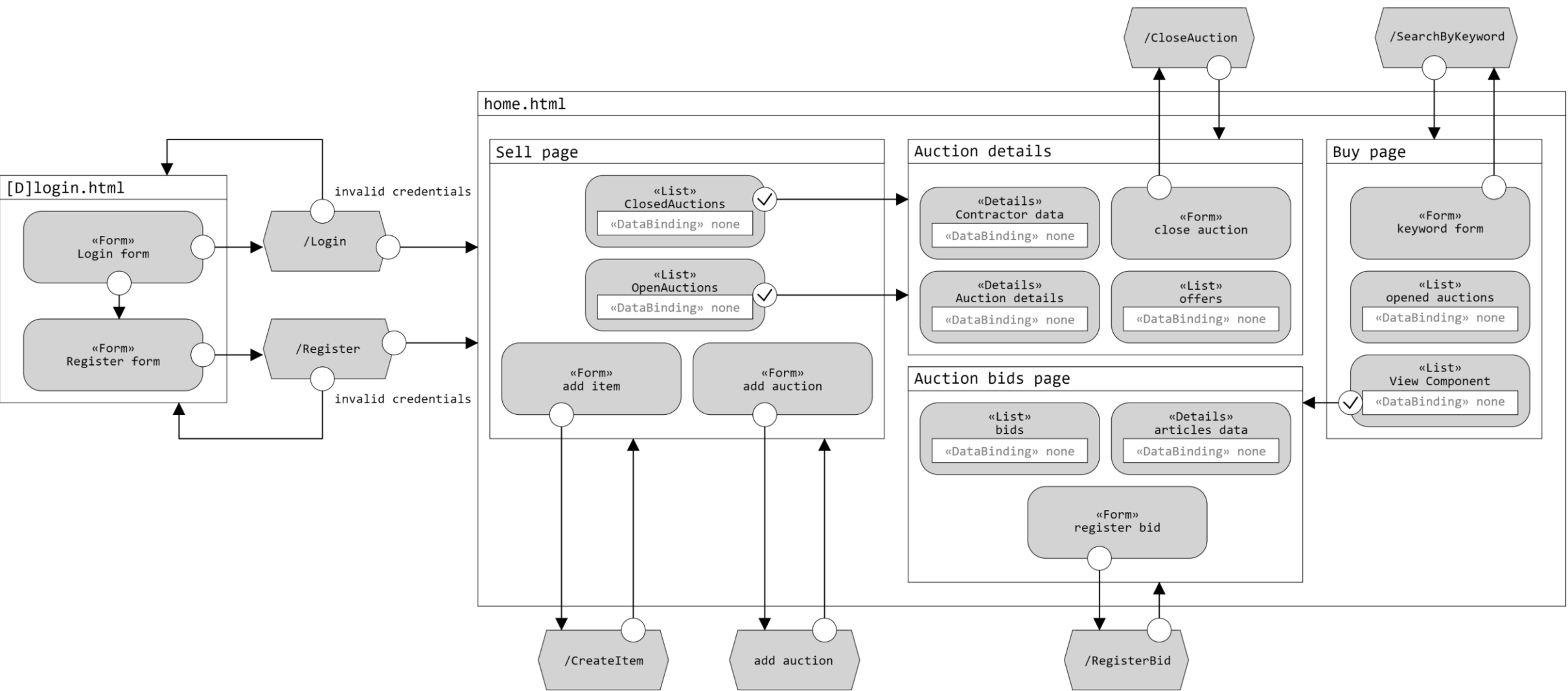
- PageOrchestrator
 - start()** : initializes page components;
 - refresh()**: displays initial content and manages it with the use of cookies;
- UserInfo
 - show()** : shows user's data in page header
- CookiesManager()
 - setCookieView()**:takes note of the last user's action was related to buy page or sell page;
 - addVisitedAuction()**:takes note of the last auctions visited by the user;
 - getCookieView()**: return the value of the toDisplay cookie;
 - getCookieVisitedAuction()**: return the value of the visitedAuction cookie;
 - removeDuplicates()**: remove duplicates from the visitedAuction cookie;
- BuyInfo
 - show()**:shows elements of buy page;
- SellInfo()
 - show()**: shows elements of sell page;
 - showItemsToSell()**:shows not auctioned items;
- AuctionBids()
 - showBids(id_auction)**:shows bids of the auction with id_auction as Id.
 - show()**:block the auctionBids container and calls showBids(id_auction).
 - clear()**: reset the auction bids container

Client side components:

Home:

- OpenedAuctions
 - show():** shows the user's opened auctions.
 - showOpenAuctionDetails(id_auction):** calls the AJAX function and both showDetails(id_auction) and showBids(id_auction).
 - showDetails(id_auction):** shows details of the auction.
 - showBids(id_auction):** shows bids of the auction.
- ClosedAuctions
 - show():** shows the user's closed auctions.
 - showClosedAuctionDetails(id_auction):** calls the AJAX function and both showDetails(id_auction) and showBuyer(id_auction).
 - showDetails(id_auction):** shows details of the auction.
 - showBuyer(id_auction):** shows details of the buyer of the auction.
- VisitedAuctionsInfo
 - show():** shows last visited auctions by the user .

IFML design



Events & Actions

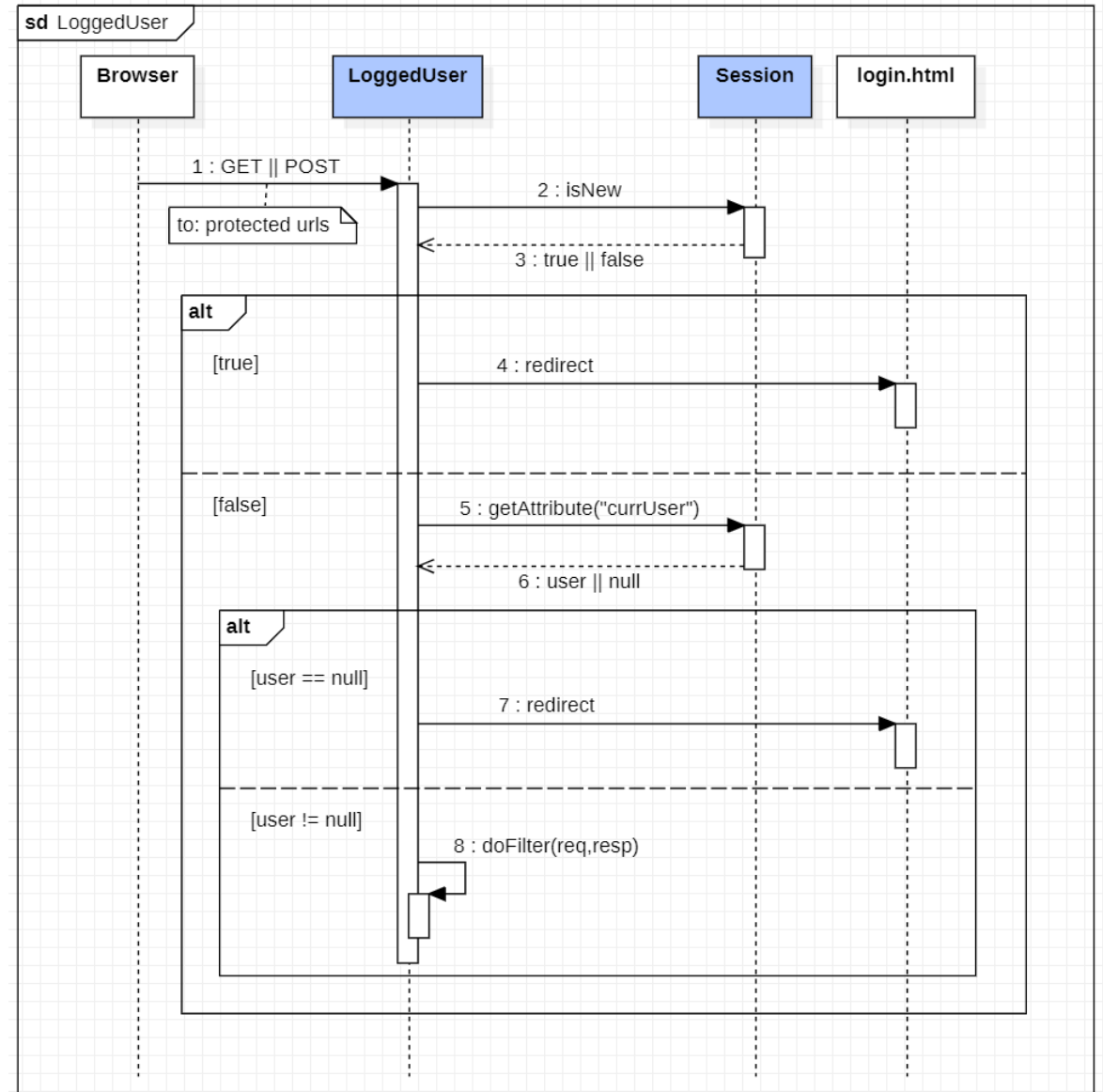
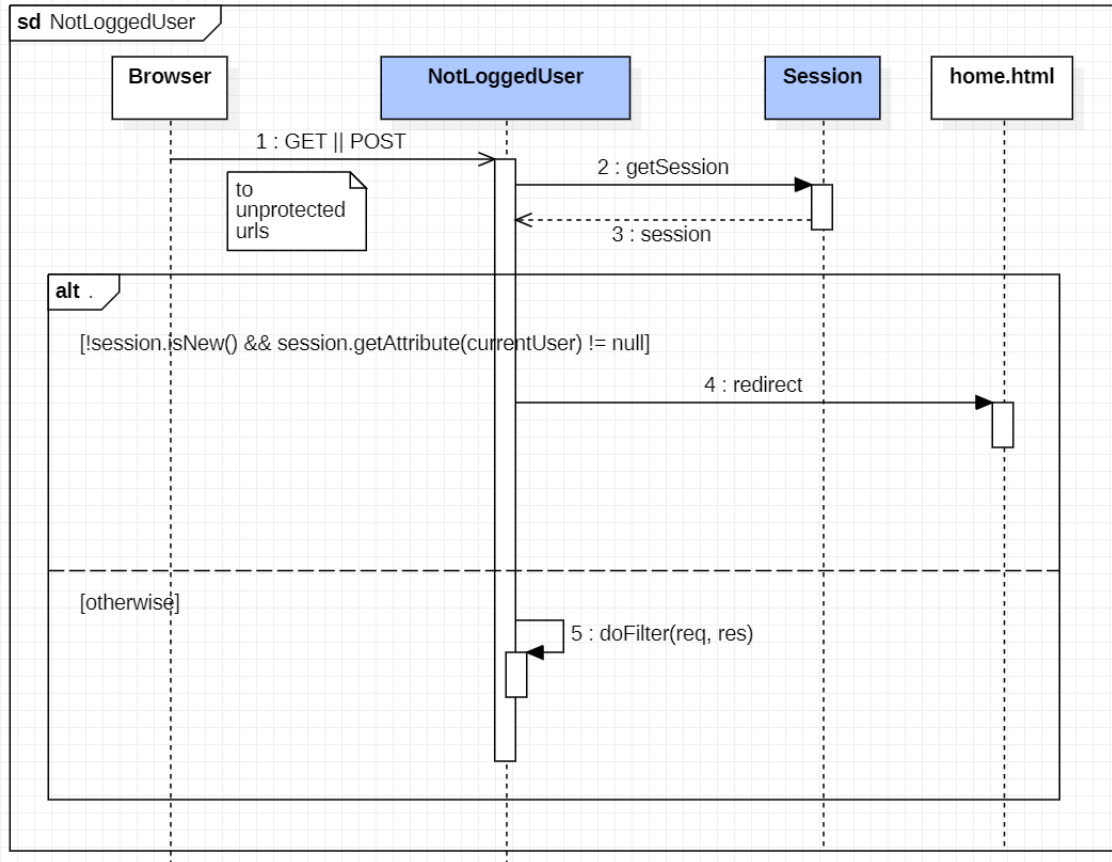
Client side		Server side	
Evento	Azione	Evento	Azione
Login.html → login form → submit	Controllo dati	POST (username password)	Controllo credenziali
Login.html → register form → submit	Controllo dati	POST (username, password, address)	Controllo credenziali
Home.html → load	Carica dati da DB e aggiorna la view	Diverse GET senza parametri	Carica dati da DB e crea contenuto HTML
Home.html → Aste aperte / chiuse → Click ID	Aggiorna view con dati asta	Diverse GET (id_auction)	Carica dati da DB e crea contenuto HTML
Home.html → Post new item → Post new item	Controllo dati	POST (item data)	Aggiunge nuovo item
Home.html → Create auction → Create new auction	Controllo dati	POST (auction data)	Aggiunge nuova asta, modifica stato a item
Home.html → Search by keyword → Search auctions	Controllo dati	GET (keyword)	Carica dati da DB e crea contenuto HTML
Home.html → Search by keyword → Click ID	Aggiorna view con dati asta	GET (id_auction)	Carica dati da DB e crea contenuto HTML
Home.html → Search by keyword → Make your offer	Controllo dati	POST (offer data)	Aggiunge nuova offerta

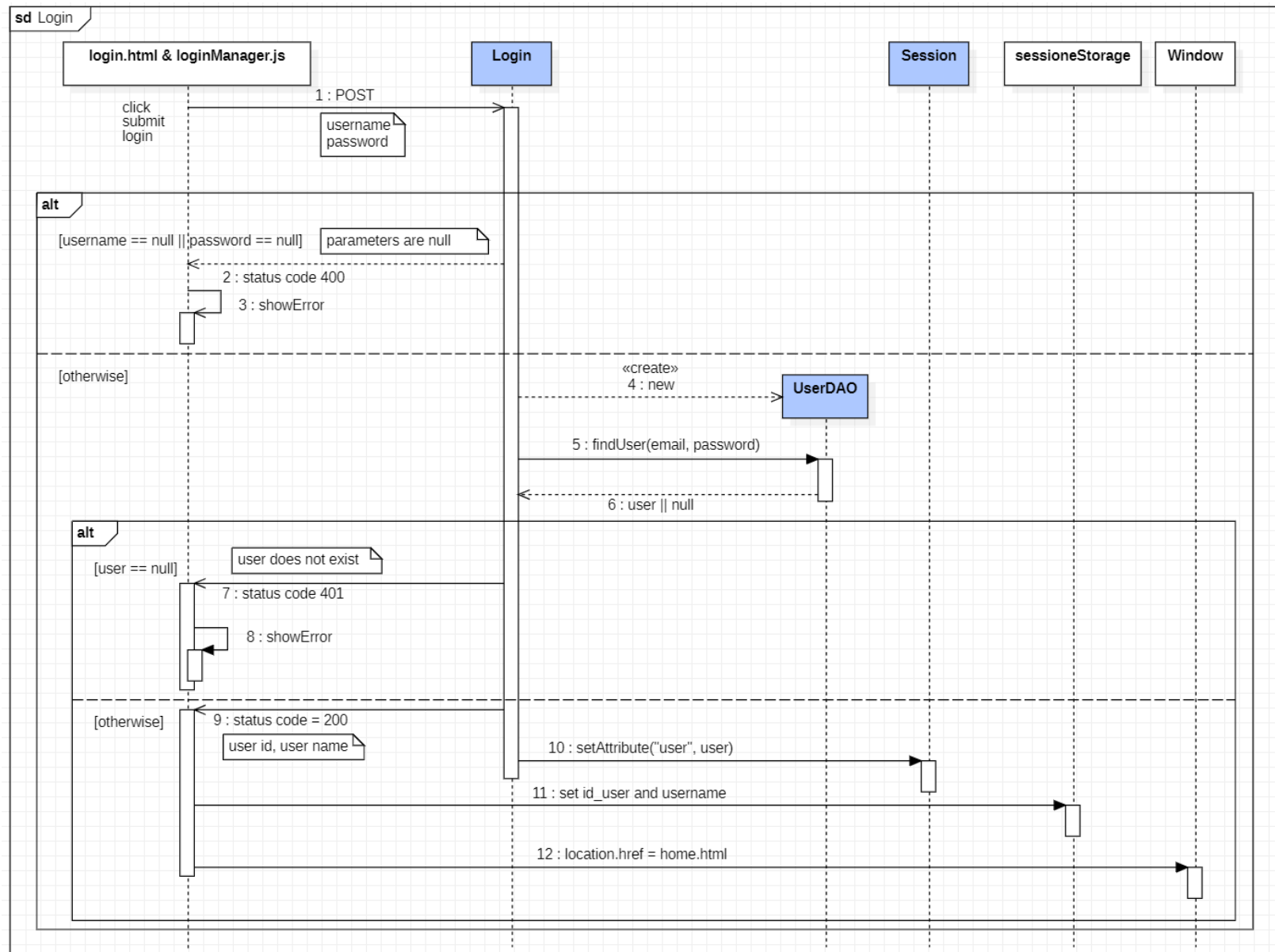
Controller & Event Handlers

Client side		Server side	
Evento	Azione	Evento	Azione
Login.html → login form → submit	Function makeCall	POST (username, password)	Login (Servlet)
Login.html → register form → submit	Function makeCall	POST (username, password, address)	Register (Servlet)
Home.html → load	Function PageOrchestrator	GET (no param)	GetAwardedAuctions(Servlet) / GetOpenedAuctions(Servlet), GetClosedAuctions(Servlet), GetNotAuctionedItems(Servlet)
Home.html → Aste aperte / chiuse → Click ID	Function makeCall	GET (id_auction)	GetAuctionDetails (Servlet)
Home.html → Post new item → Post new item	Function makeCall	POST (item data)	CreateItem (Servlet)
Home.html → Create auction → Create new auction	Function makeCall	POST (auction data)	CreateAuction (Servlet)
Home.html → Search by keyword → Search auctions	Function makeCall	GET (keyword)	SearchByKeyword (Servlet)
Home.html → Search by keyword → Click ID	Function makeCall	GET (id_auction)	GetAuctionBids (Servlet)
Home.html → Search by keyword → Make your offer	Function makeCall	POST (bid data)	RegisterBid (Servlet)

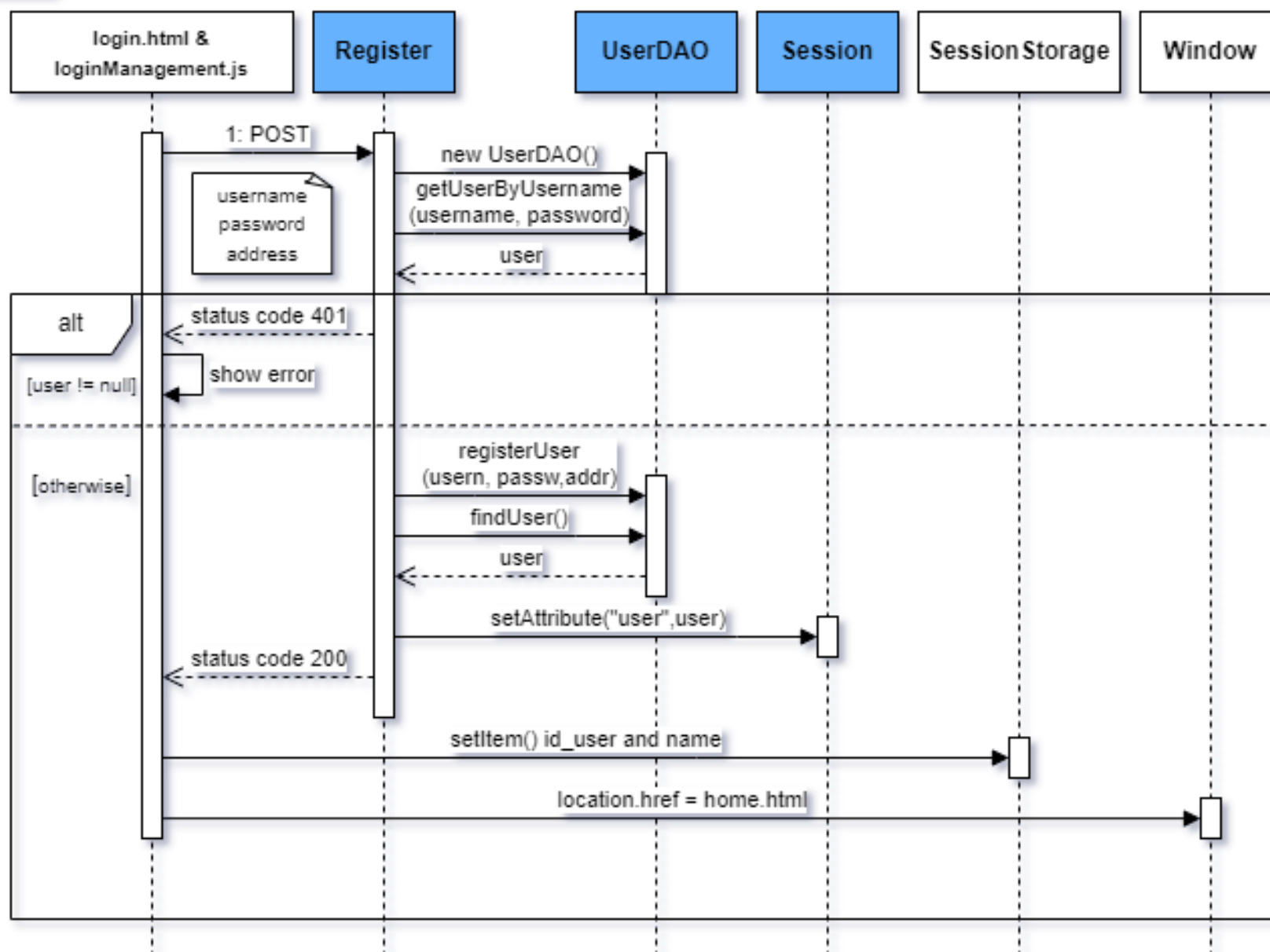
Sequence diagrams

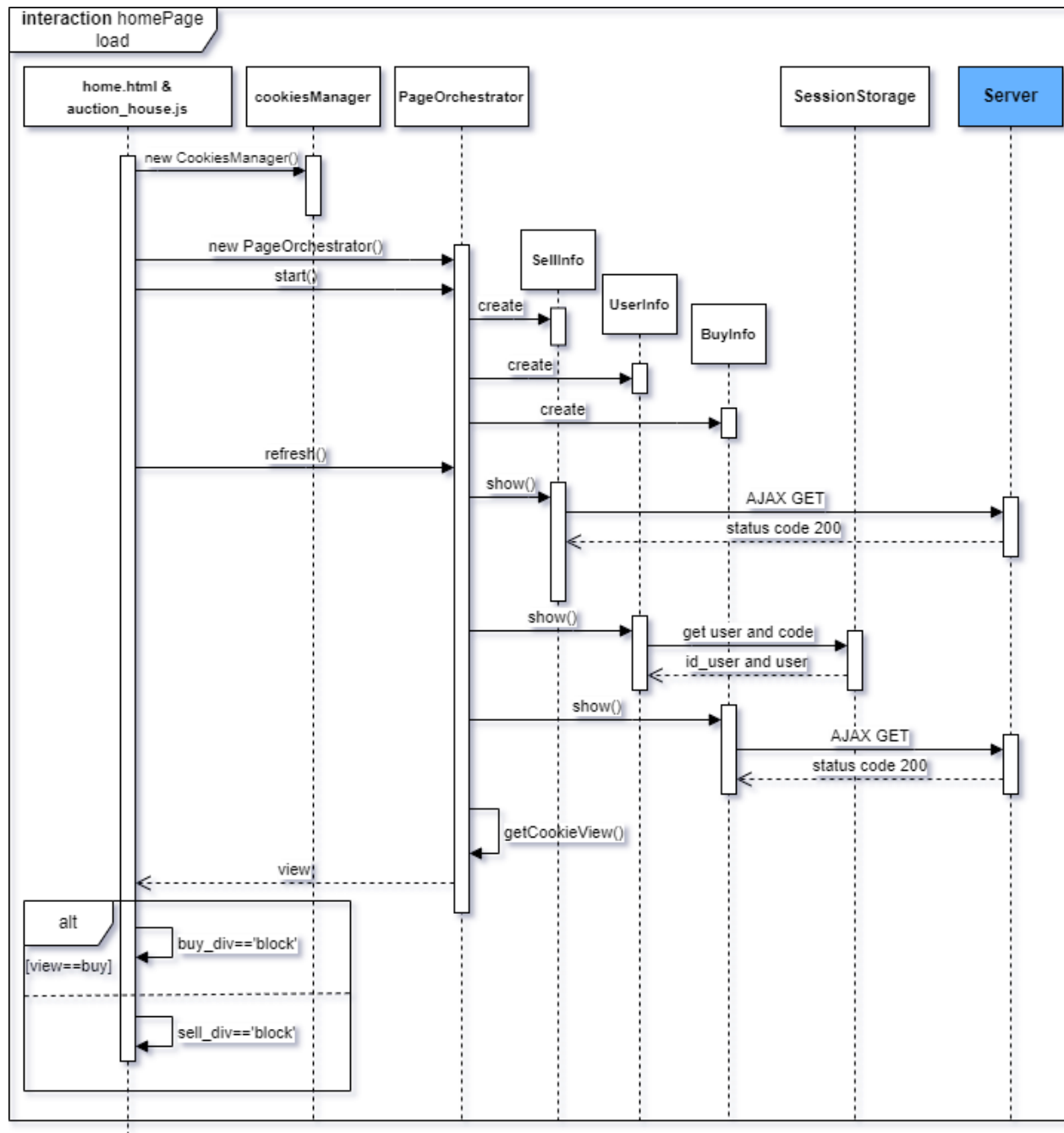
For simplicity, the controls on the input parameters have been omitted. Each request to the server passes through one of two filters (LoggedUser if the user is already logged in, otherwise NotLoggedUser), which check if the session is valid and if the user is logged in.



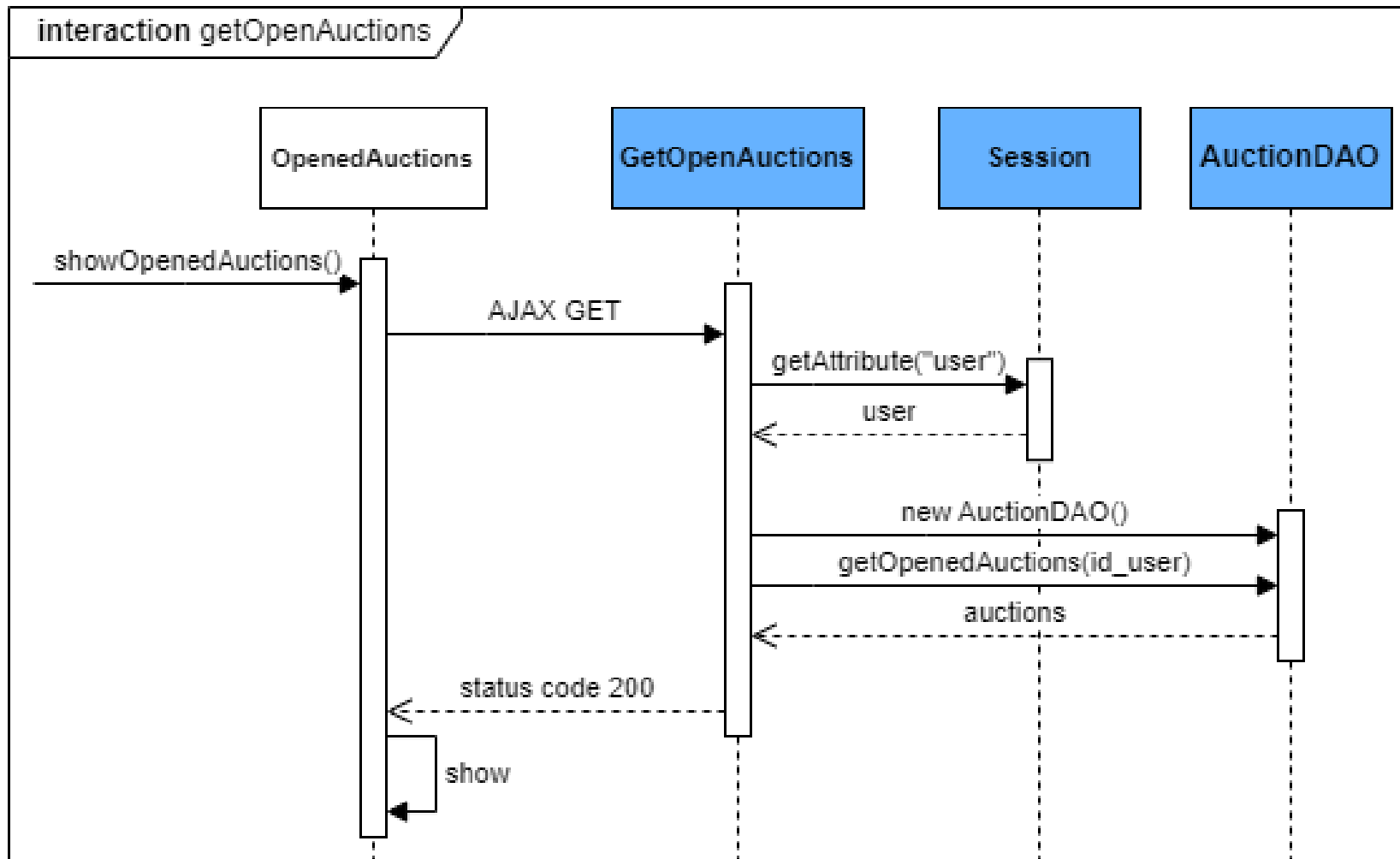


sd Register



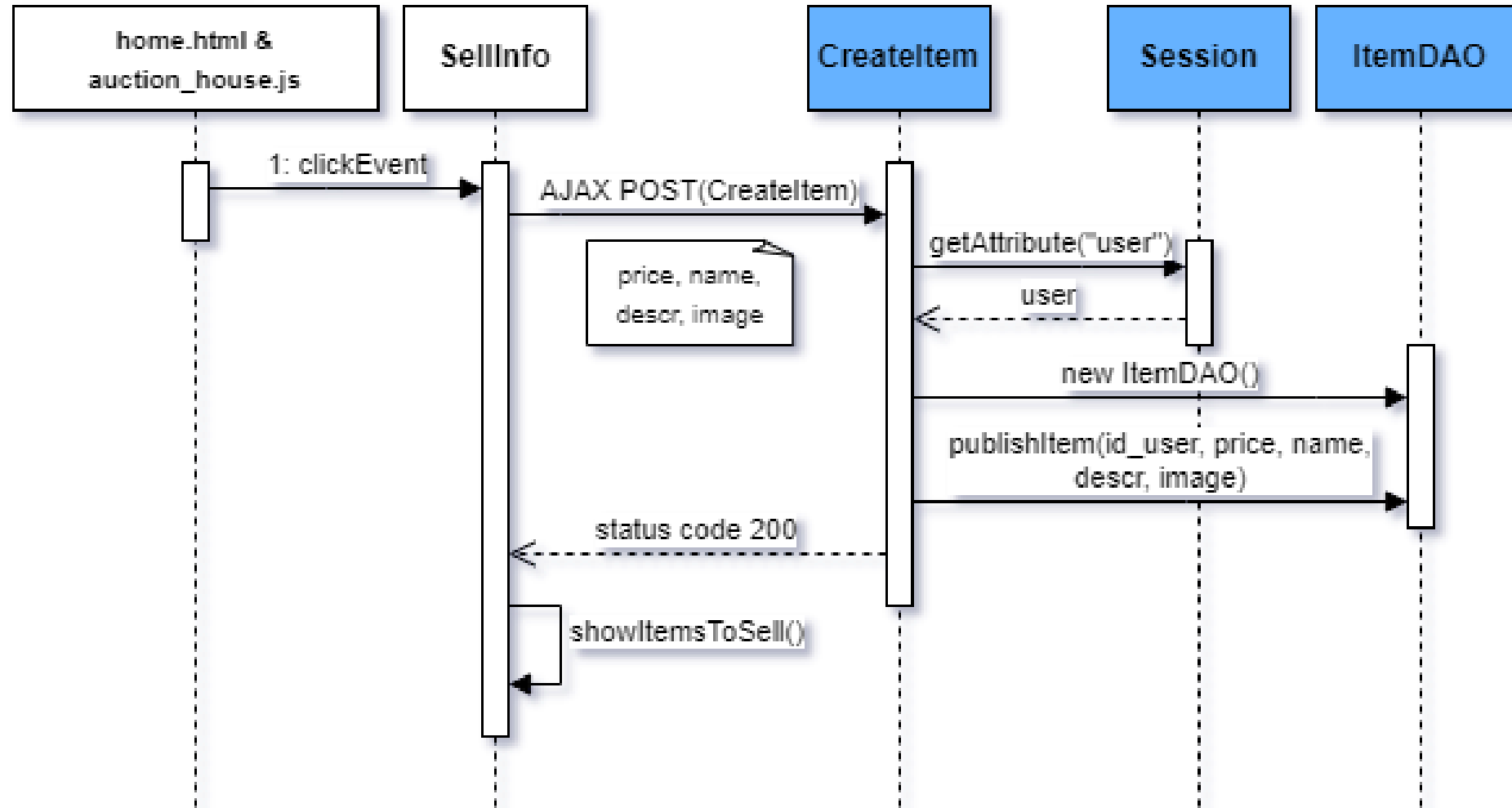


Ajax calls are detailed in the next sequence diagrams

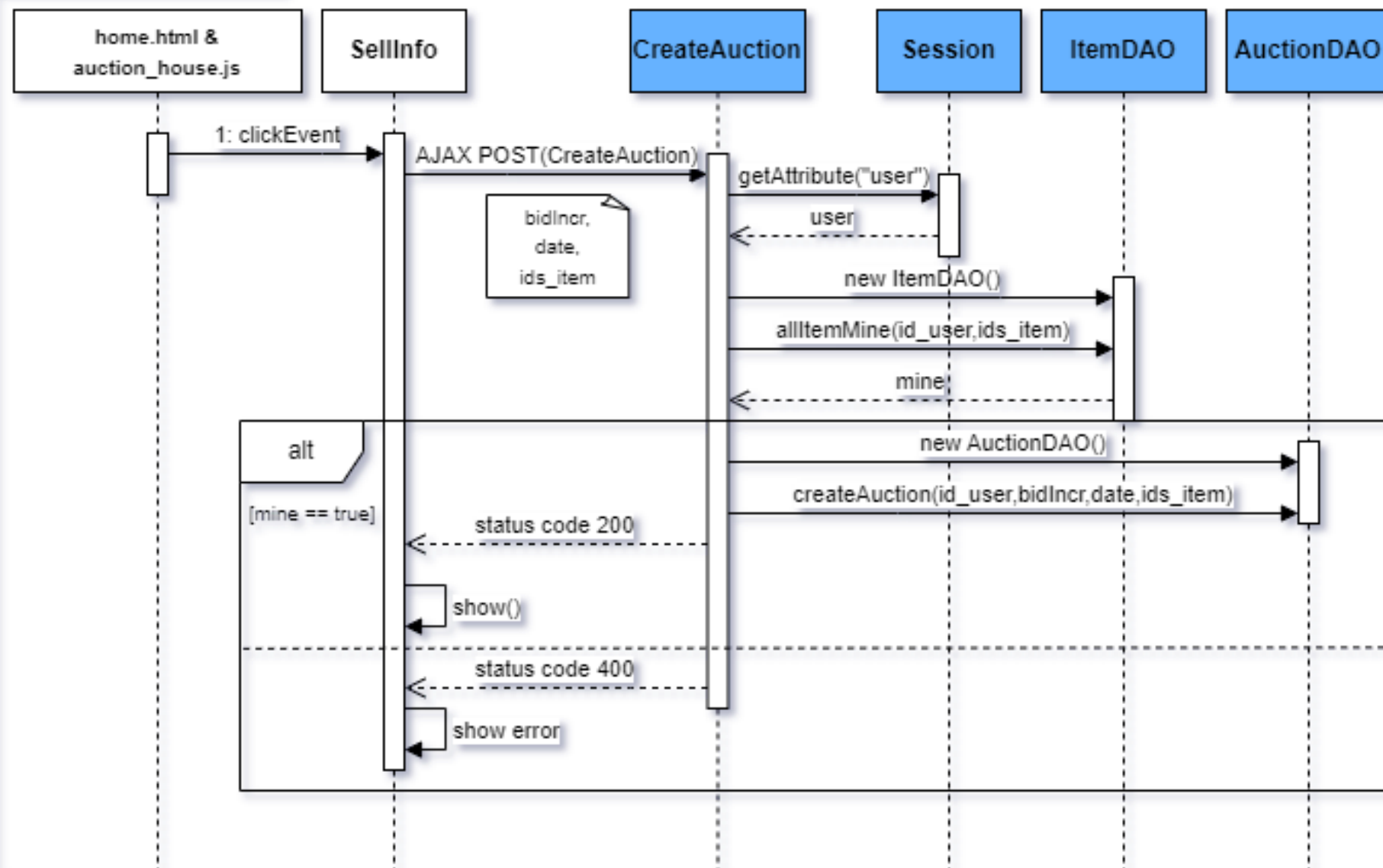


every server call involving a "get" servlet looks like this. For this reason all the sequence diagrams of the «Gets» have been omitted

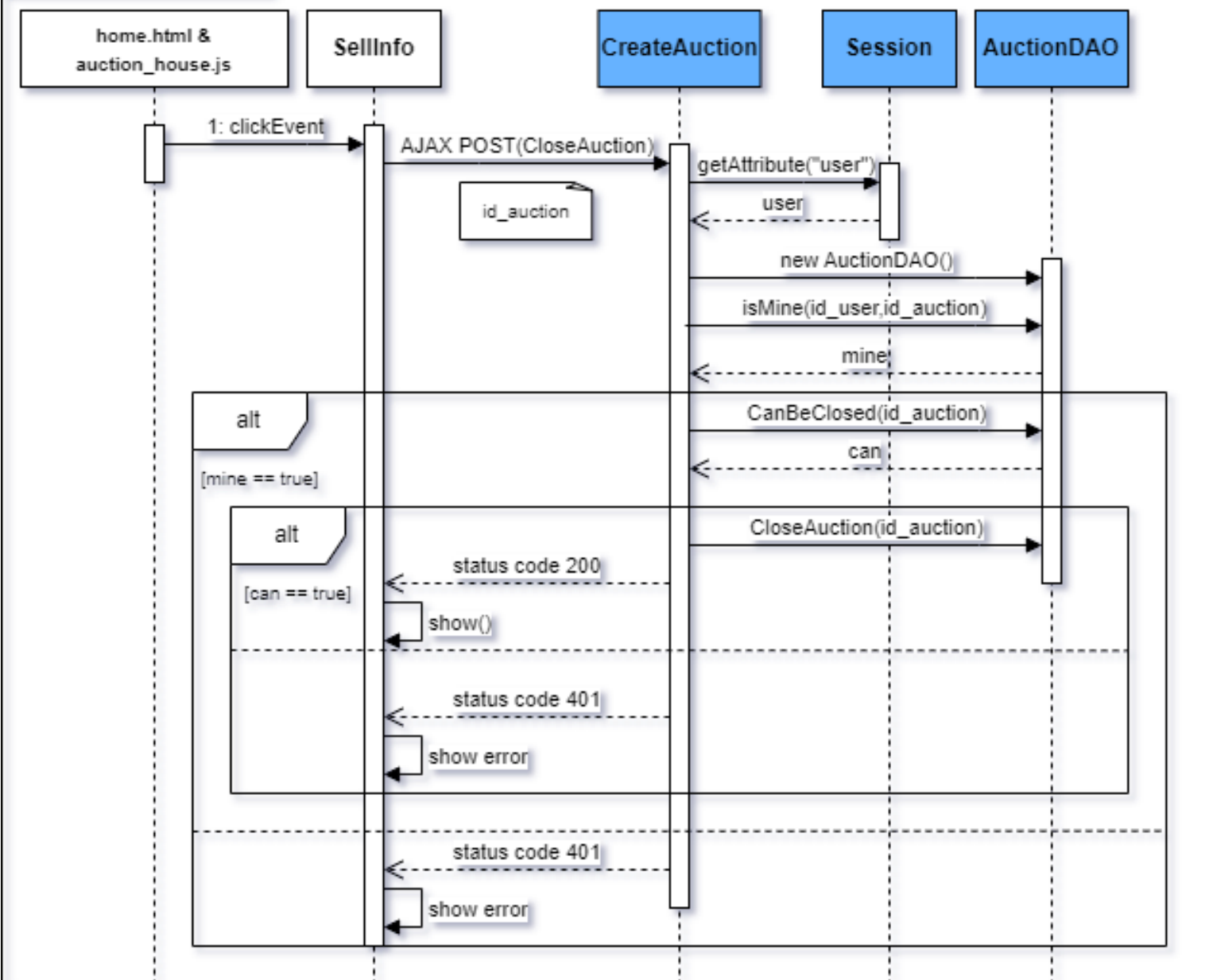
interaction CreateItem

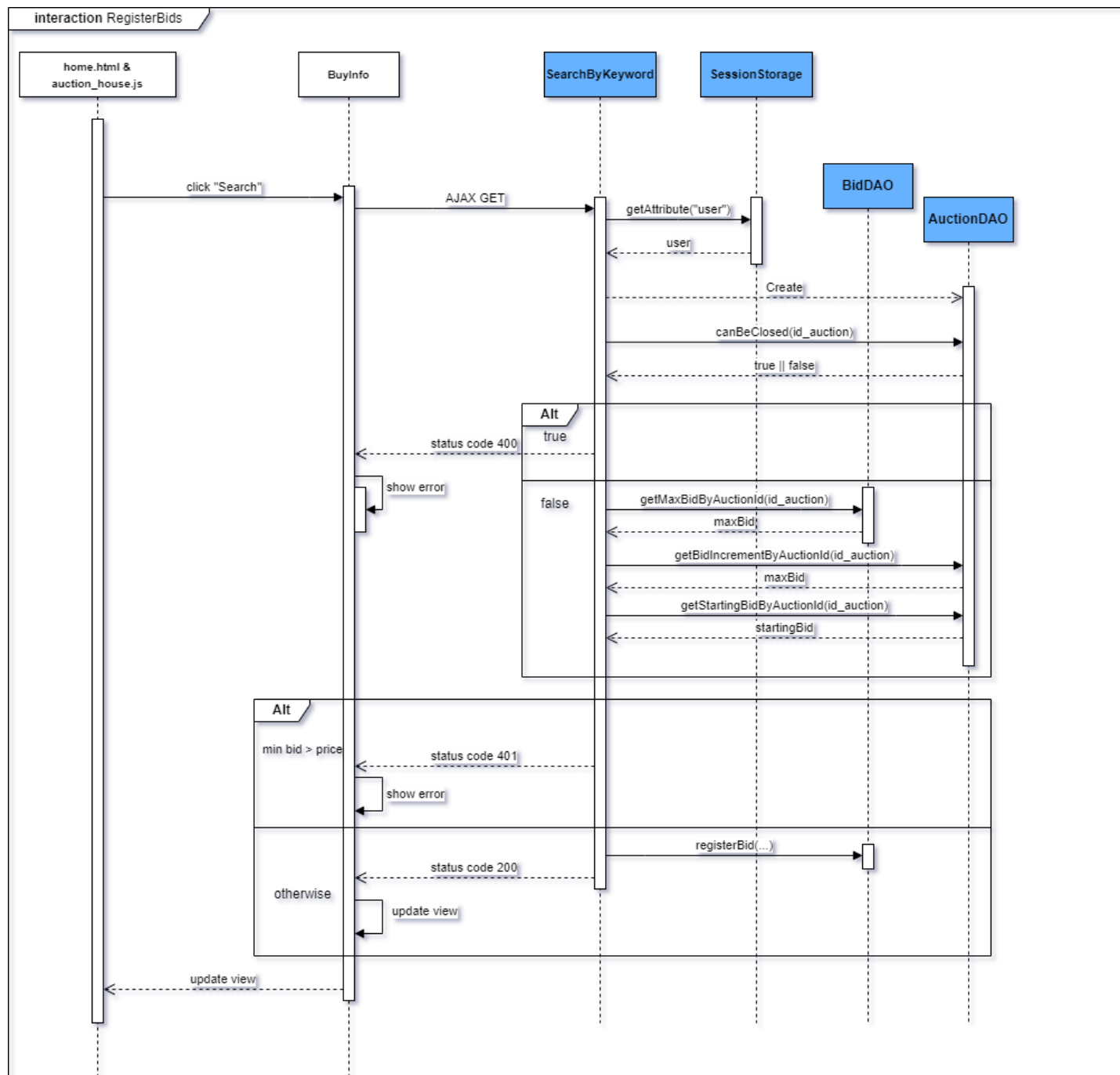


interaction CreateAuction

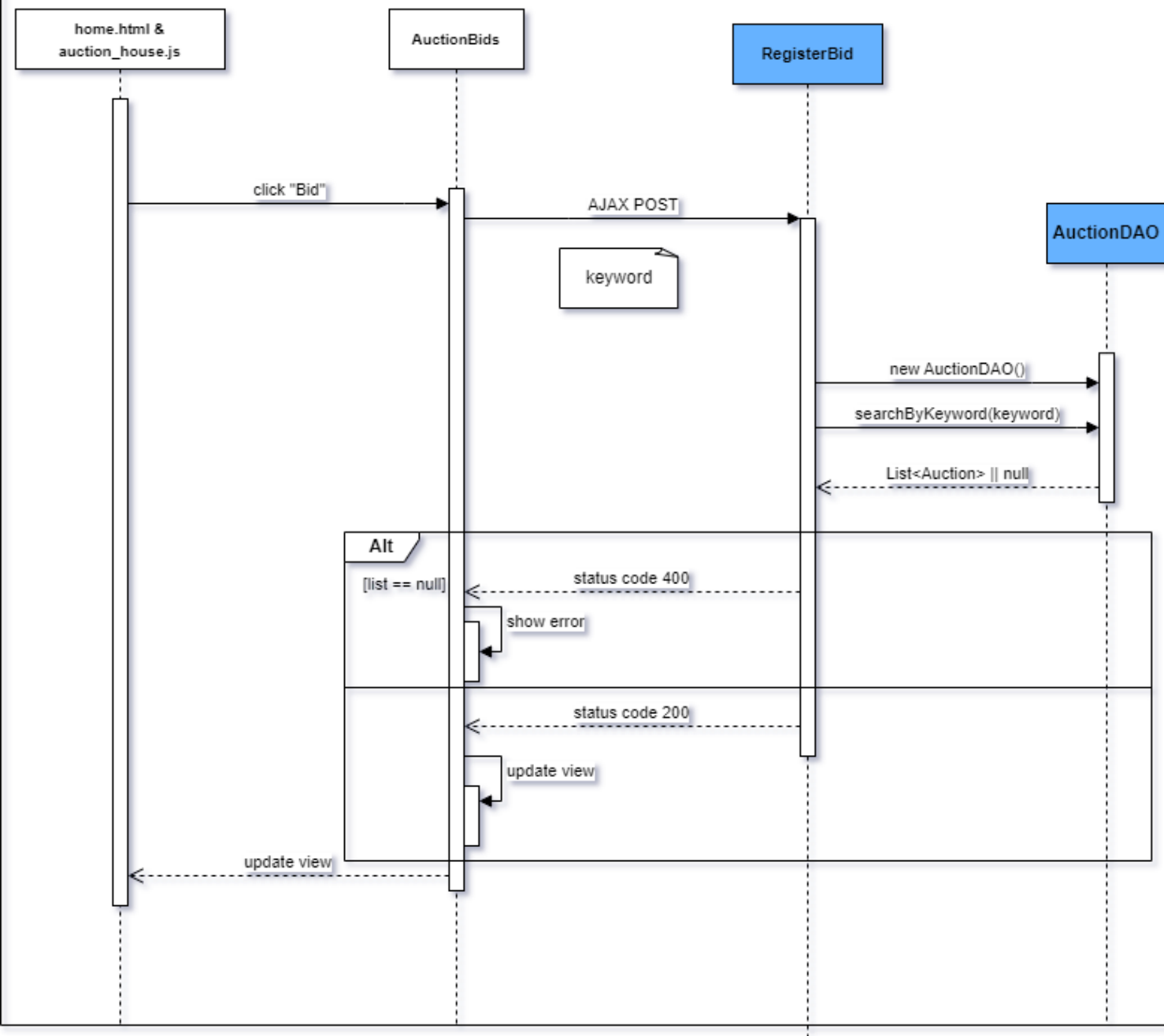


interaction CloseAuction





interaction searchByKeyword



sd Logout

